# Multithreaded Architectures and Programming on Modern Processors: the Multithreaded Version

Hung-Wei Tseng

# Recap: Five implementations

- Which of the following implementations will perform the best on modern pipeline processors?

A

```
inline int popcount(uint64_t x){
  int c=0;
  while(x)  {
        c += x & 1;
        x = x >> 1;
    }
    return c;
}
```

B

```
inline int popcount(uint64_t x) {
   int c = 0;
   while(x)     {
     c += x & 1;
     x = x >> 1;
     c += x & 1;
     x = x >> 1;
     c += x & 1;
     x = x >> 1;
     c += x & 1;
     x = x >> 1;
   }
   return c;
}
```

C

```
inline int popcount(uint64_t x) {
     int c = 0;
     int table[16] = {0, 1, 1, 2, 1,
2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4};
     while(x)     {
         c += table[(x & 0xF)];
         x = x >> 4;
     }
     return c;
}
```

D

```
inline int popcount(uint64_t x) {
     int c = 0;
     int table[16] = {0, 1, 1, 2, 1,
2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4};
     for (uint64_t i = 0; i < 16; i++)
     {
         c += table[(x & 0xF)];
         x = x >> 4;
     }
     return c;
 }
```

E

```
inline int popcount(uint64_t x) {
     int c = 0;
     for (uint64_t i = 0; i < 16; i++)
     {
         switch((x & 0xF))
         {
             case 1: c+=1; break;
             case 2: c+=1; break;
             case 3: c+=2; break;
             case 4: c+=1; break;
             case 5: c+=2; break;
             case 6: c+=2; break;
             case 7: c+=3; break;
             case 8: c+=1; break;
             case 9: c+=2; break;
             case 10: c+=2; break;
             case 11: c+=3; break;
             case 12: c+=2; break;
             case 13: c+=3; break;
             case 14: c+=3; break;
             case 15: c+=4; break;
             default: break;
         }
         x = x >> 4;
     }
     return c;
}
```

# Tips of programming on modern processors

- Minimize the critical path operations
  - Don’t forget about optimizing cache/memory locality first!
    - Memory latencies are still way longer than any arithmetic instruction
    - Can we use arrays/hash tables instead of lists?
  - Branch can be expensive as pipeline get deeper
    - Sorting
    - Loop unrolling
  - Still need to carefully avoid long latency operations (e.g., mod)
- Since processors have multiple functional units — code must be able to exploit instruction-level parallelism
  - Hide as many instructions as possible under the “critical path”
  - Try to use as many different functional units simultaneously as possible
- Modern processors also have accelerated instructions
- Compiler can do fairly go optimizations, but with limitations

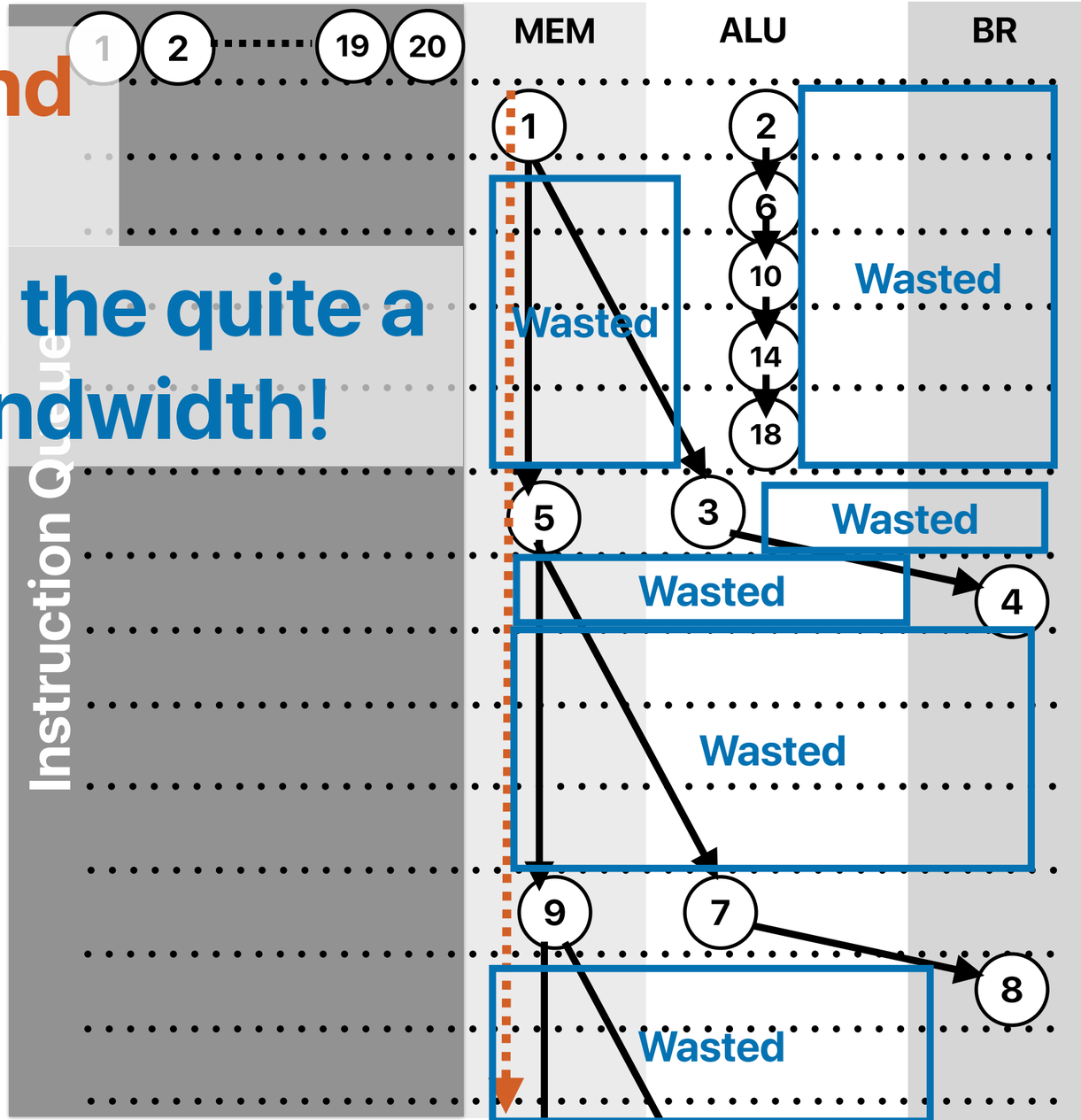
# What if we have “unlimited” fetch/issue width — “linked list”

## Even unlimited issue width and a perfect cache cannot help!

## We’re wasting the quite a lot issue bandwidth!

```
do {
    number_of_nodes++;
    current = current->next;
} while ( current != NULL );
```

```
① .L3:    movq    8(%rdi), %rdi
②         addl    $1, %eax
③         testq   %rdi, %rdi
④         jne     .L3
```



# Summary of popcounts

| | ET | IC | IPC/ILP | # of branches | Branch mis-prediction rate |
|---|---|---|---|---|---|
| A | 22.21 | 332 Trillions | 2.88 | 65 Trillions | 1.13% |
| B | 12.29 | 287 Trillions | 4.52 | 17 Trillions | 0.04% |
| C | 5.01 | 102 Trillions | 3.95 | 17 Trillions | 0.04% |
| D | 3.73 | 80 Trillions | 4.13 | 1 Trillions | ~0% |
| E | 54.4 | 173 Trillions | 0.61 | 44 Trillions | 18.6% |
| SSE4.2 | 1.57 | 22 Trillions | 2.7 | 1 Trillions | ~0% |

Best at 4.5

Best performing one at 2.7

# Outline

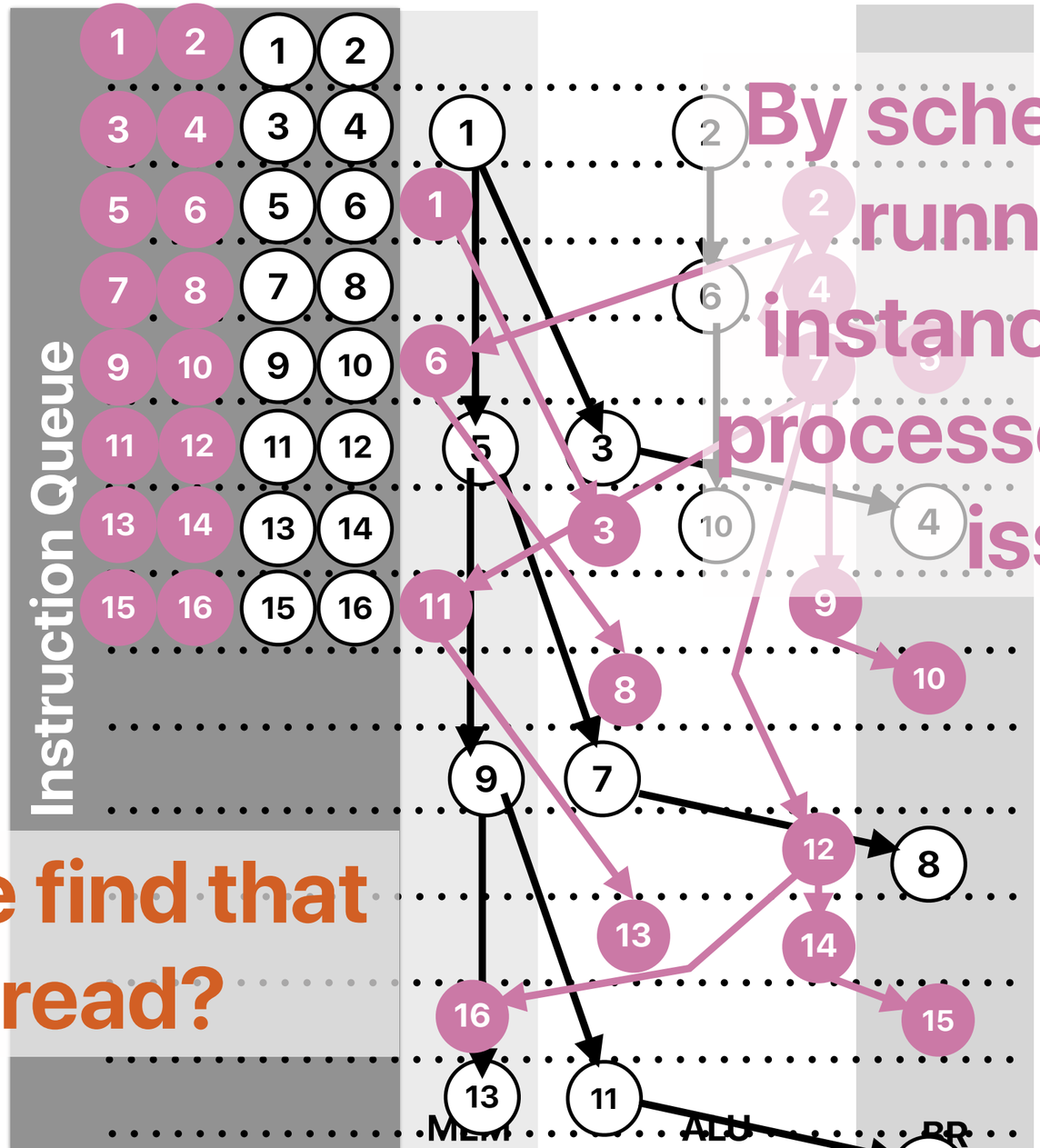
- Parallel architectures
  - Simultaneous multithreading (SMT)
  - Chip multiprocessors (CMP)
- Parallel programming

# Parallel architectures

# Simultaneous multithreading

# Concept: Simultaneous Multithreading (SMT)

```
① movq    8(%rdi), %rdi
② addl    $1, %eax
③ testq   %rdi, %rdi
④ jne     .L3
⑤ movq    8(%rdi), %rdi
⑥ addl    $1, %eax
⑦ testq   %rdi, %rdi
⑧ jne     .L3
⑨ movq    8(%rdi), %rdi
⑩ addl    $1, %eax
⑪ testq   %rdi, %rdi
⑫ jne     .L3
```




```
① movl (%rdi), %ecx
② cmpq %rdx, %rdi
③ addl %ecx, %eax
④ jne  .L3
⑤ movl (%rdi), %ecx
⑥ addq $4, %rdi
⑦ addl %ecx, %eax
⑧ cmpq %rdx, %rdi
⑨ jne  .L3
⑩ movl (%rdi), %ecx
⑪ addq $4, %rdi
⑫ addl %ecx, %eax
⑬ cmpq %rdx, %rdi
⑭ jne  .L3
⑮ movl (%rdi), %ecx
⑯ addq $4, %rdi
```

**By scheduling another running program instance (thread), the processor has 0 wasted issue slots!**

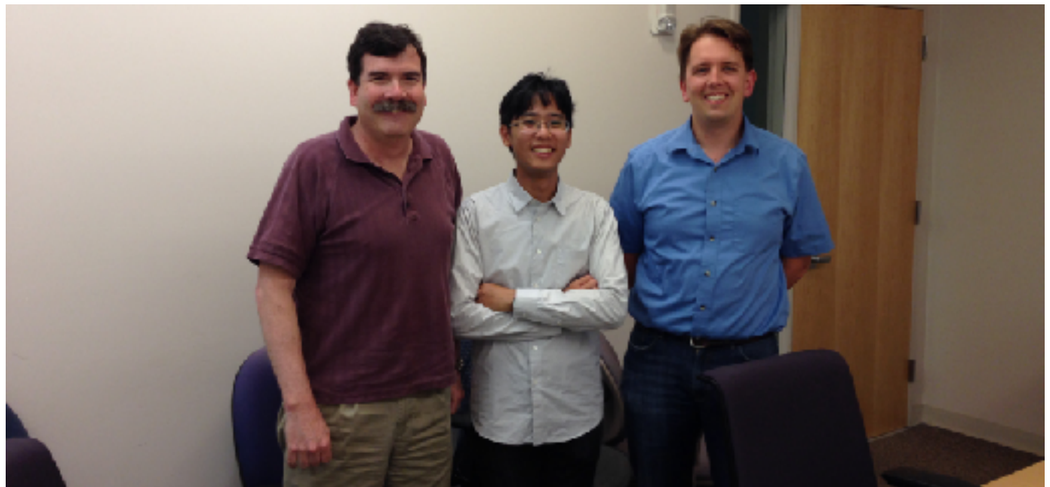
**Where can we find that "other" thread?**

# Where to find another “thread”

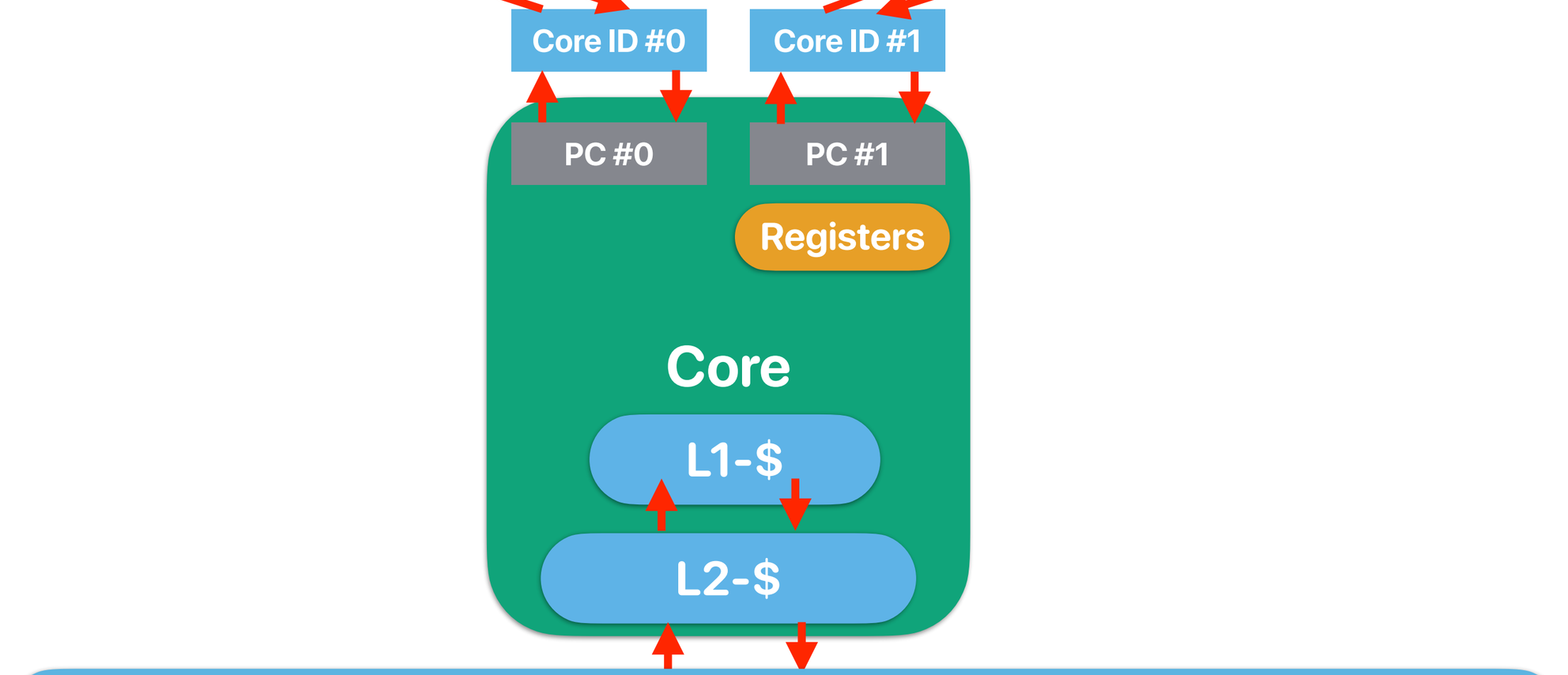
- Another process/running program forked from a completely different program
- Another process forked/cloned from the current process
  - The forked program cloned the memory content at the forked time
  - The forked program cannot view the memory space of the original process
  - The forked program can perform a subset of tasks form the original process
  - The forked and the original process can exchange data through files or inter-process communication APIs
- A software thread spawned from the current process
  - The spawned thread executes a function instance from the main process to offload computation from the main process
  - The spawned thread shares the memory space with the main process

# Simultaneous multithreading

- The processor can schedule instructions from different threads/processes/ programs
- Fetch instructions from different threads/processes to fill the not utilized part of pipeline
  - Exploit “thread level parallelism” (TLP) to solve the problem of insufficient ILP in a single thread
  - You need to create an illusion of multiple processors for OSs
- Invented by Dean Tullsen (Now a professor at **UCSD CSE**)



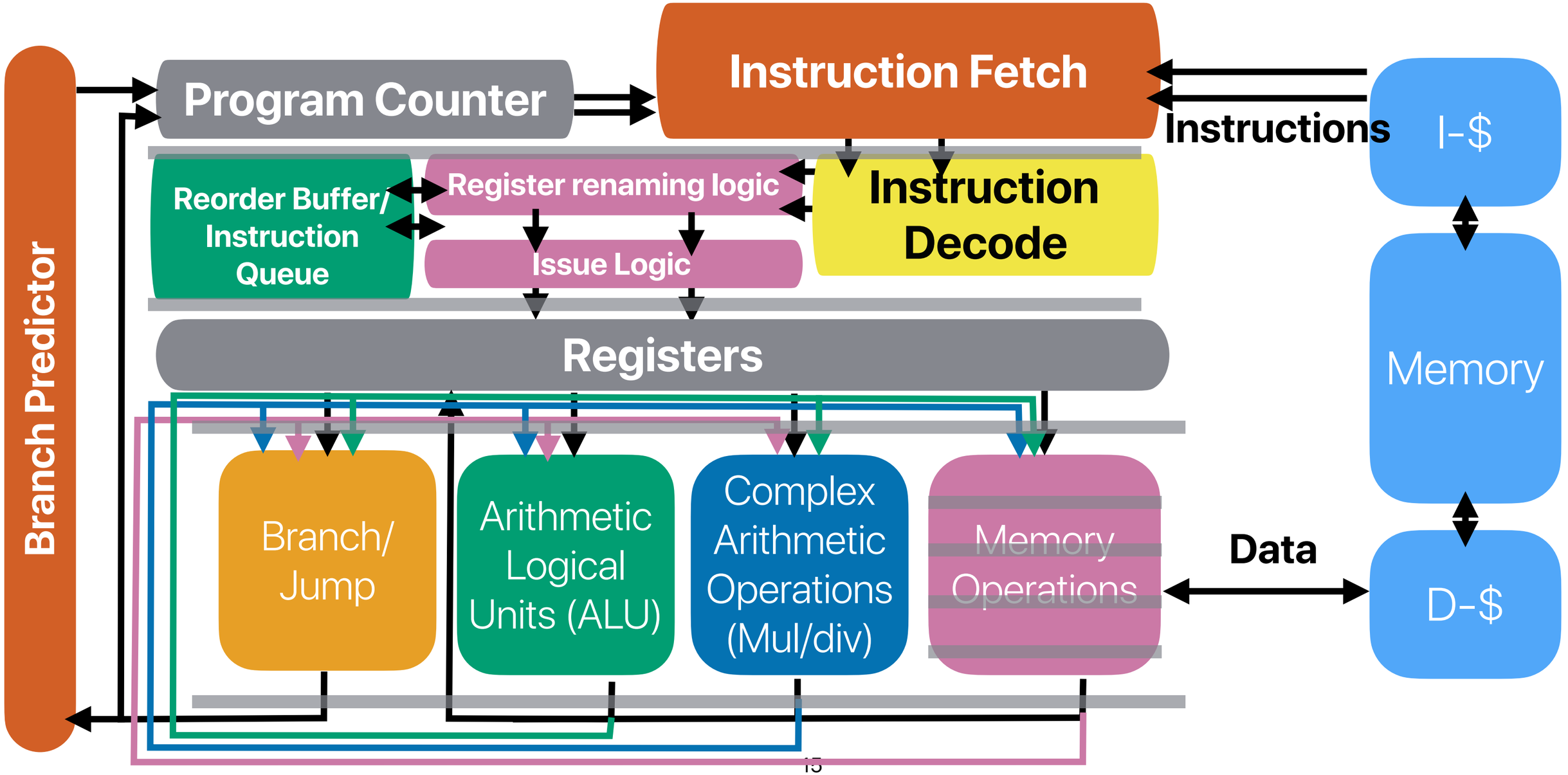
# SMT from the user/OS' perspective



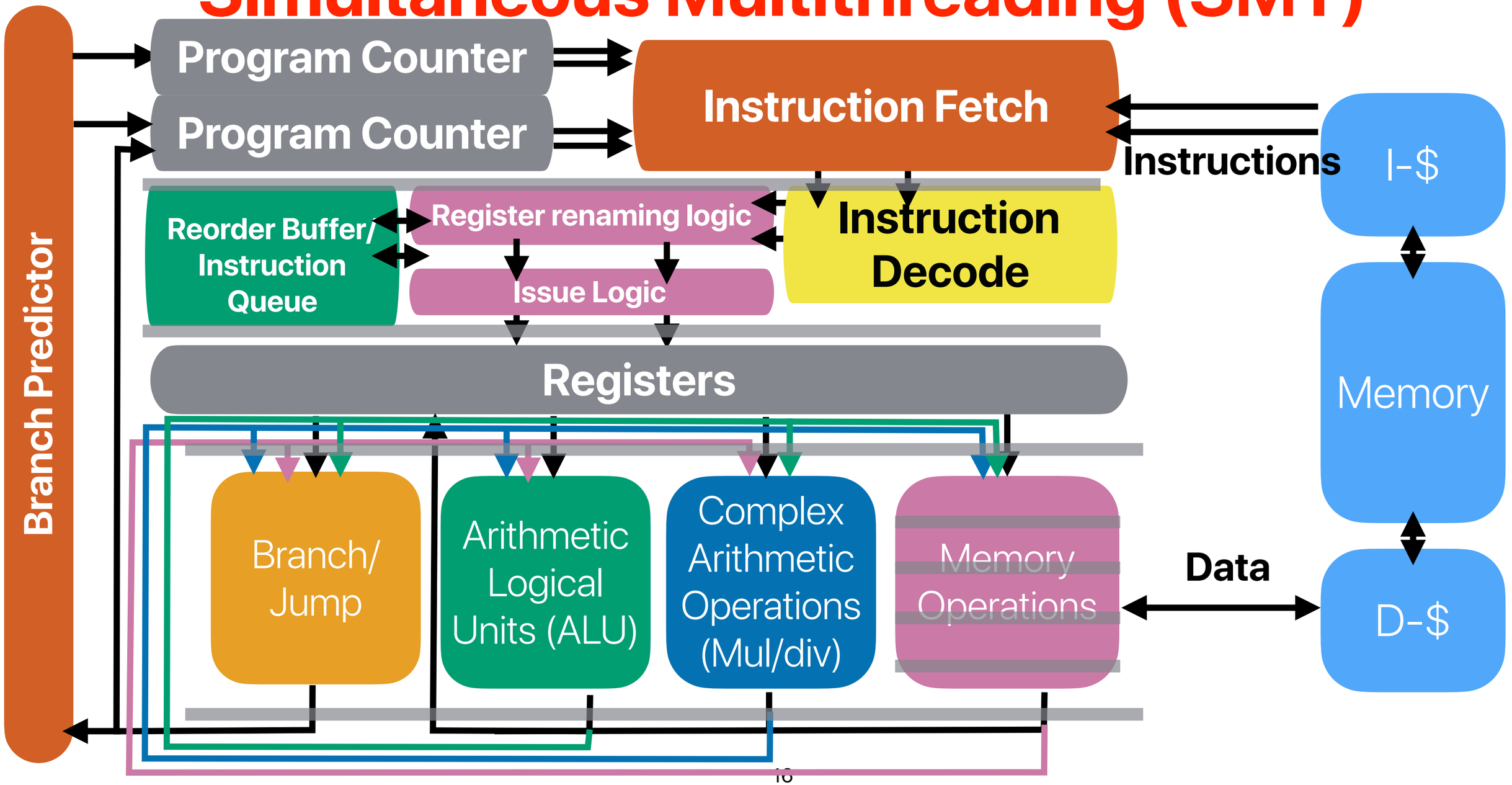
# How do we support two running programs in one pipeline?

- We need two program counters
- We need two sets of architectural to physical register mappings
- We do not need
  - Duplicated cache — virtually indexed, physically tagged cache already addressed that
  - Duplicated pipeline functional units — isn’t sharing the whole purpose?
  - Duplicated reorder buffer — you simply need to tag which process the instruction belongs to

# Recap: Register renaming



# Simultaneous Multithreading (SMT)



https://www.pollev.com/hungweitseng close in 1:30

# Pros/Cons of SMT

- How many of the following statements about SMT is/are correct?
  - ① SMT makes processors with deep pipelines more tolerable to mis-predicted branches
  - ② SMT can improve the throughput of a single-threaded application
  - ③ SMT processors can better utilize hardware during cache misses comparing with superscalar processors with the same issue width
  - ④ SMT processors can have higher cache miss rates comparing with superscalar processors with the same cache sizes when executing the same set of applications.

  A. 0
  B. 1
  C. 2
  D. 3
  E. 4

# Pros/Cons of SMT

- How many of the following statements about SMT is/are correct?
  - ① SMT makes processors with deep pipelines more tolerable to mis-predicted branches
  - ② SMT can improve the throughput of a single-threaded application
  - ③ SMT processors can better utilize hardware during cache misses comparing with superscalar processors with the same issue width
  - ④ SMT processors can have higher cache miss rates comparing with superscalar processors with the same cache sizes when executing the same set of applications.
  - A. 0
  - B. 1
  - C. 2
  - D. 3
  - E. 4

# Pros/Cons of SMT

- How many of the following statements about SMT is/are correct?
  - ① ✓ SMT makes processors with deep pipelines more tolerable to mis-predicted branches
  - ② SMT can ~~improve~~ the throughput of a single-threaded application
  - ③ ✓ SMT processors can better utilize hardware during cache misses comparing with superscalar processors with the same issue width
  - ④ ✓ SMT processors can have higher cache miss rates comparing with superscalar processors with the same cache sizes when executing the same set of applications.

  A. 0
  B. 1
  C. 2
  D. 3
  E. 4

We can execute from other threads/contexts instead of the current one

hurt, b/c you are sharing resource with other threads.

We can execute from other threads/ contexts instead of the current one

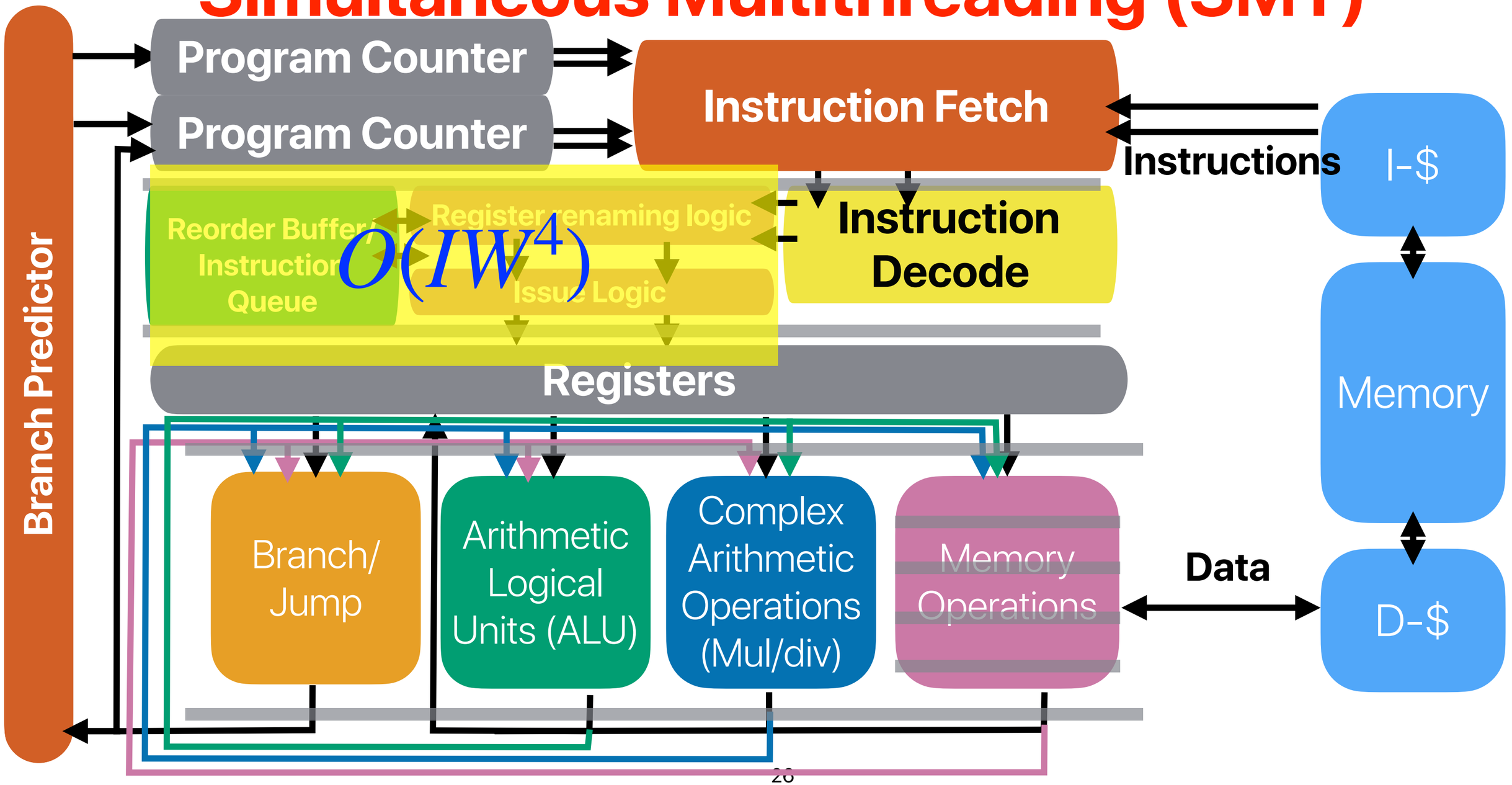
b/c we're sharing the cache

```
Architecture:                    x86_64
CPU op-mode(s):                  32-bit, 64-bit
Byte Order:                      Little Endian
Address sizes:                   39 bits physical, 48 bits virtual
CPU(s):                          8
On-line CPU(s) list:             0-7
Thread(s) per core:              2
Core(s) per socket:              4
Socket(s):                       1
NUMA node(s):                    1
Vendor ID:                       GenuineIntel
CPU family:                      6
Model:                           151
Model name:                      12th Gen Intel(R) Core(TM) i3-12100F
Stepping:                        5
CPU MHz:                         3300.000
CPU max MHz:                     5500.0000
CPU min MHz:                     800.0000
BogoMIPS:                        6604.80
Virtualization:                  VT-x
L1d cache:                       192 KiB
L1i cache:                       128 KiB
```

# SMT in real practice

- Intel HyperThreading (supports up to two threads per core)
  - Intel Pentium 4, Intel Atom, All Intel Core i7s and Core i9s
- AMD RyZen (Zen microarchitecture)
- If you see a processor with “threads” more than “cores”, that must because of SMT!

# Simultaneous Multithreading (SMT)



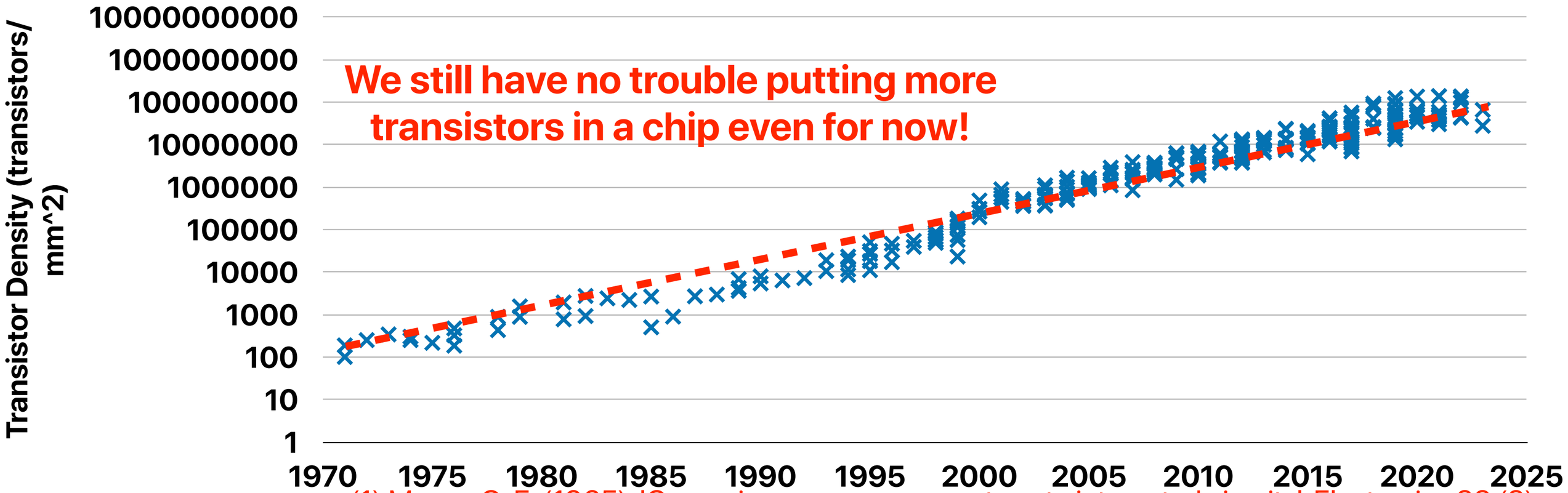
# Takeaways: parallel architectures

- SMT processors can better utilize the pipeline resources by allowing simultaneous execution of multiple threads
  - Improved execution throughput
  - May hurt the latency of each thread since we share functional units & cache

# Chip-Multiprocessors (CMP) or Multi-core processors

# Recap: Moore's Law(1)

- The number of transistors we can build in a fixed area of silicon doubles every 12 ~ 24 months.
- Moore's Law "was" the most important driver for historic CPU performance gains



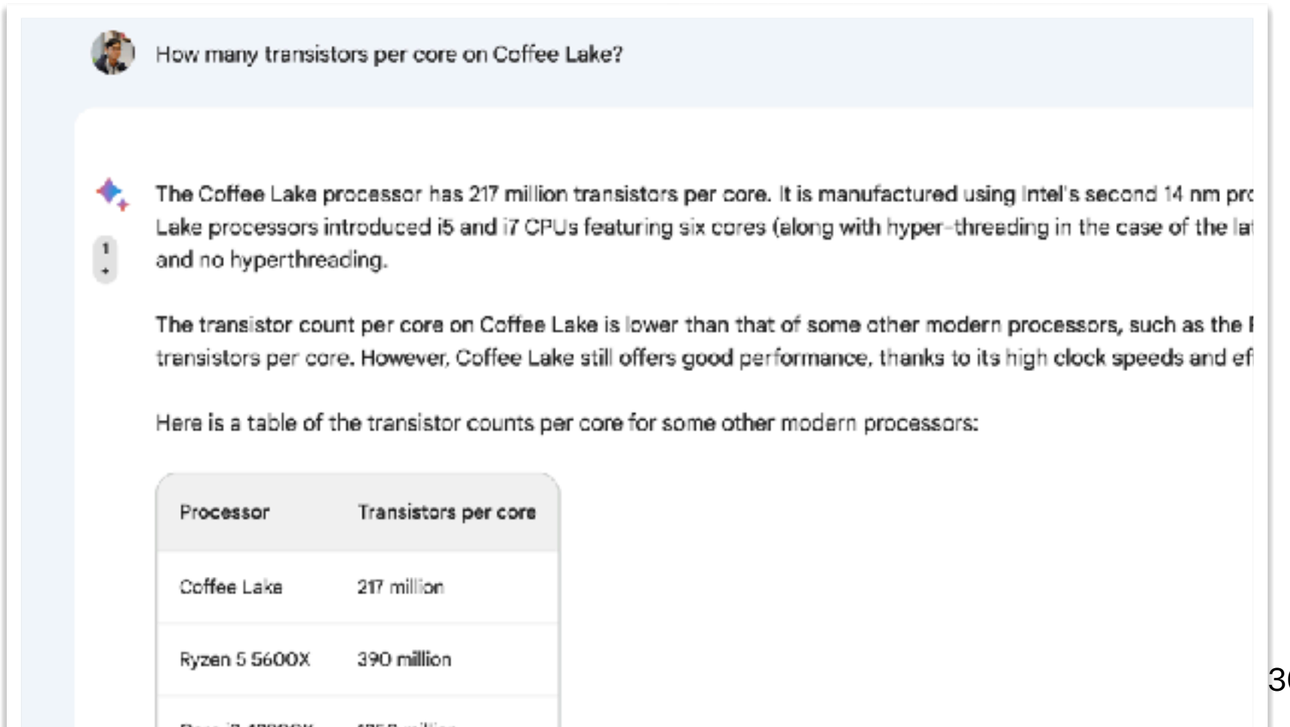

(1) Moore, G. E. (1965), 'Cramming more components onto integrated circuits', Electronics 38 (8) .

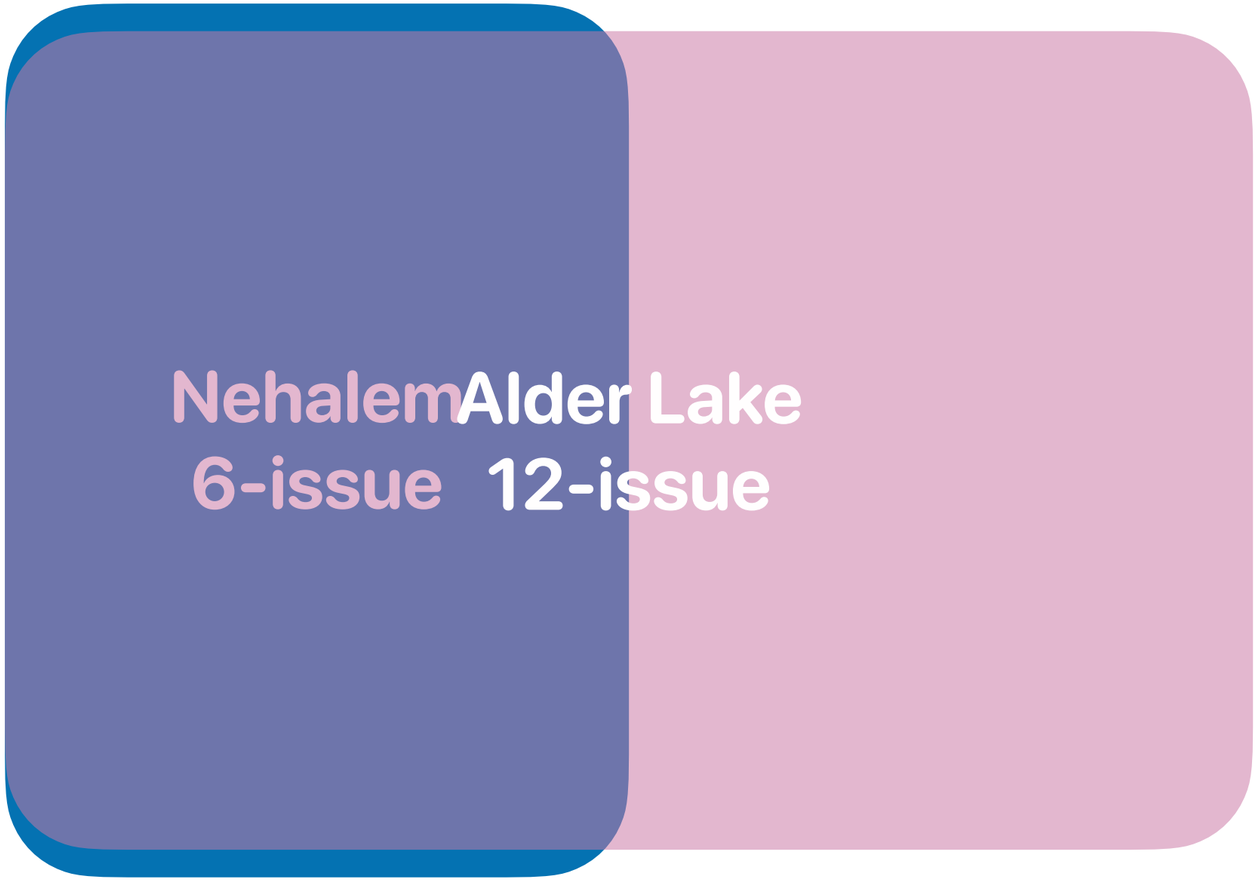


Plot based on https://en.wikipedia.org/wiki/Transistor_count by Hung-Wei Tseng

# Transistor Counts

| Microarchitecture | Transistor Count | Issue-width | Year |
|---|---|---|---|
| Alder Lake | 325 M | 5x ALU, 7x Memory | 2021 |
| Coffee Lake | 217 M | 4x ALU, 4x Memory | 2017 |
| Sandy Bridge | 290 M | 3x ALU, 3x Memory | 2011 |
| Nehalem | 182.75 M | 3x ALU, 3x Memory | 2008 |



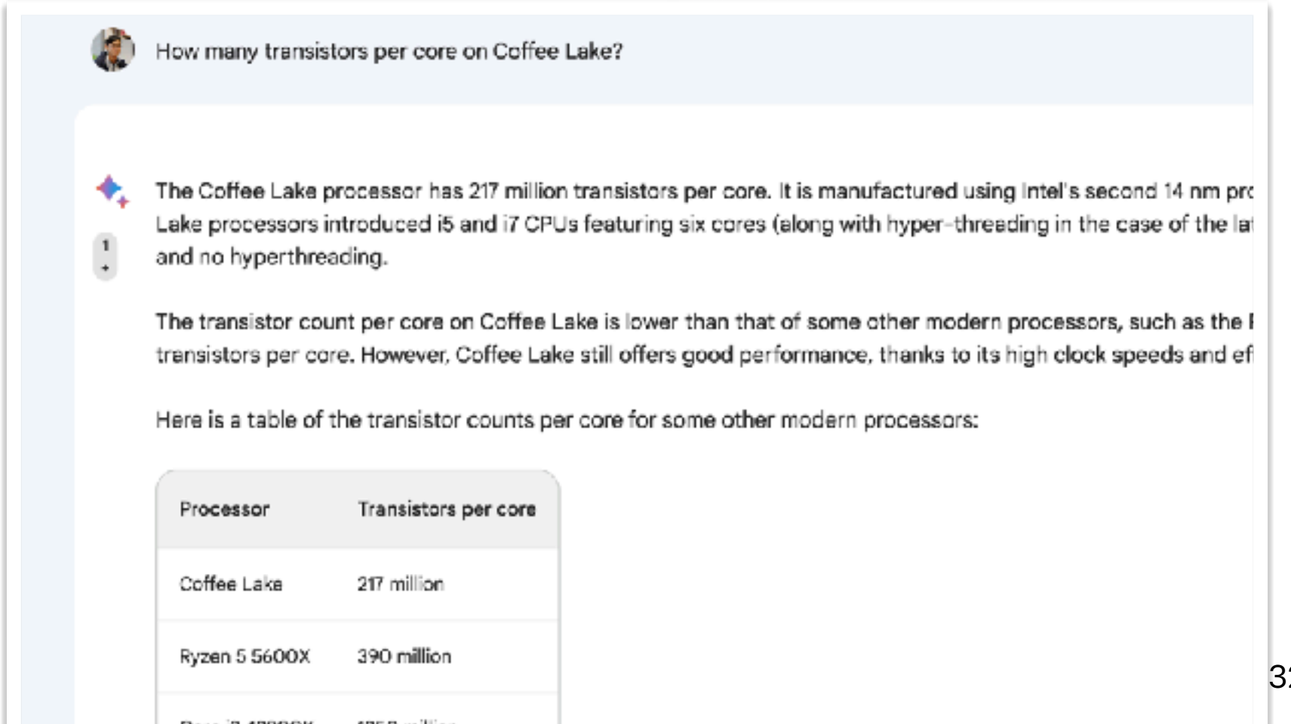

Based on https://en.wikipedia.org/wiki/Transistor_count

# Recap: do we really need very wide issue processors?

| | ET | IC | IPC/ILP | # of branches | Branch mis-prediction rate |
|---|---|---|---|---|---|
| A | 22.21 | 332 Trillions | 2.88 | 65 Trillions | 1.13% |
| B | 12.29 | 287 Trillions | 4.52 | 17 Trillions | 0.04% |
| C | 5.01 | 102 Trillions | 3.95 | 17 Trillions | 0.04% |
| D | 3.73 | 80 Trillions | 4.13 | 1 Trillions | ~0% |
| E | 54.4 | 173 Trillions | 0.61 | 44 Trillions | 18.6% |
| SSE4.2 | 1.57 | 22 Trillions | 2.7 | 1 Trillions | ~0% |

# One powerful core or two "good enough" cores?

| Microarchitecture | Transistor Count | Issue-width | Year |
|---|---|---|---|
| Alder Lake | 325 M | 5x ALU, 7x Memory | 2021 |
| Coffee Lake | 217 M | 4x ALU, 4x Memory | 2017 |
| Sandy Bridge | 290 M | 3x ALU, 3x Memory | 2011 |
| Nehalem | 182.75 M | 3x ALU, 3x Memory | 2008 |



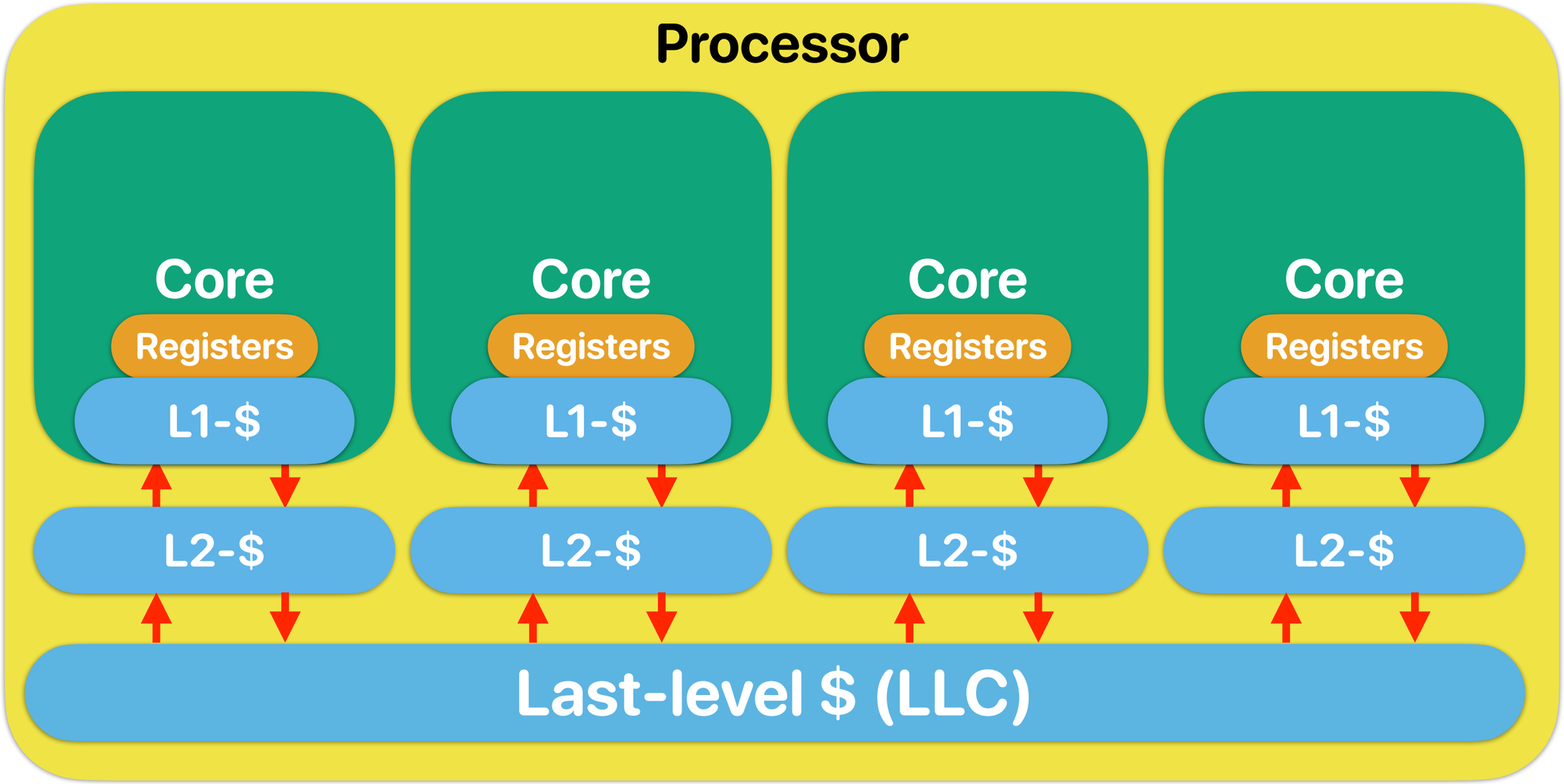

## 2x 3-issue ALUs Nehalem

Nehalem 6-issue | Alder Lake 12-issue | Nehalem 6-issue

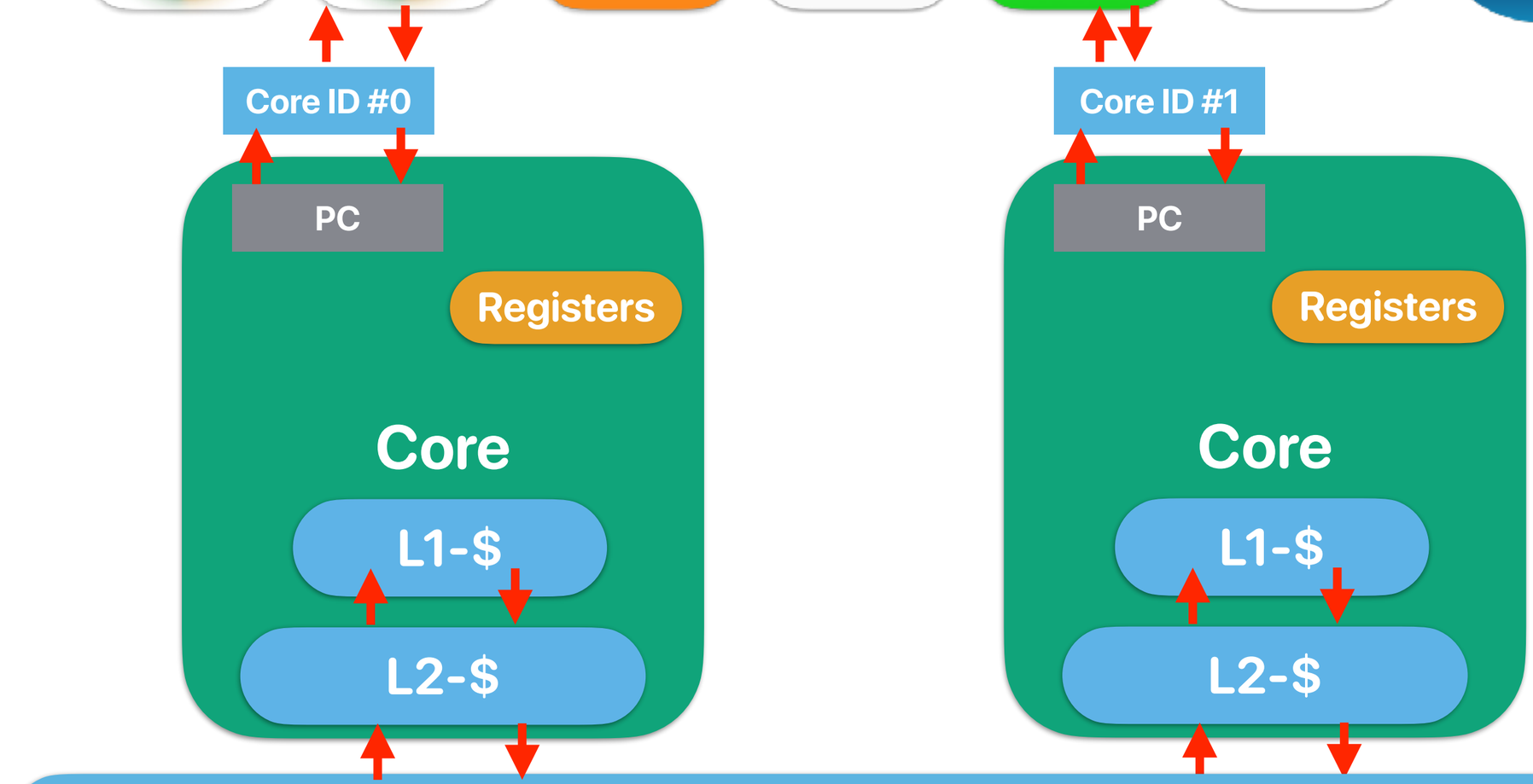
## 1x 5-issue ALUs Alder Lake

Based on https://en.wikipedia.org/wiki/Transistor_count

# Concept of CMP



# CMP from the user/OS' perspective



https://www.pollev.com/hungweitseng close in 1:30

# SMT v.s. CMP

- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
  - ① If we are just running one program in the system, the program will perform better on an SMT processor
  - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
  - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor
  - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor
  - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor

  A. 1
  B. 2
  C. 3
  D. 4
  E. 5

# SMT v.s. CMP

- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
  - ① If we are just running one program in the system, the program will perform better on an SMT processor
  - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
  - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor
  - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor
  - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor

  A. 1
  B. 2
  C. 3
  D. 4
  E. 5

# SMT v.s. CMP

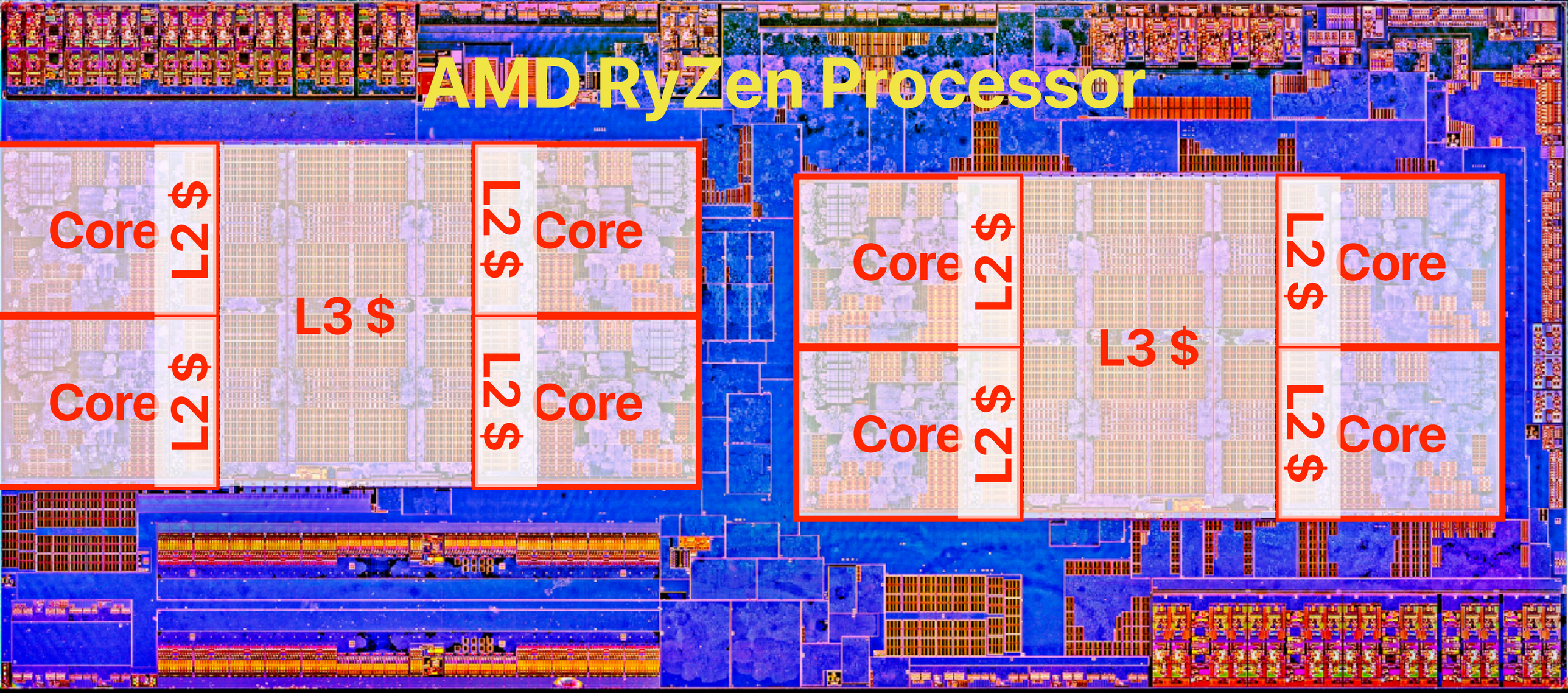
- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
  - ① If we are just running one program in the system, the program will perform better on an SMT processor — you have more resources for the program
  - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
  - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor — it depends!
  - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor — it depends! — it depends!
  - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor — it depends!

  A. 1
  B. 2
  C. 3
  D. 4
  E. 5

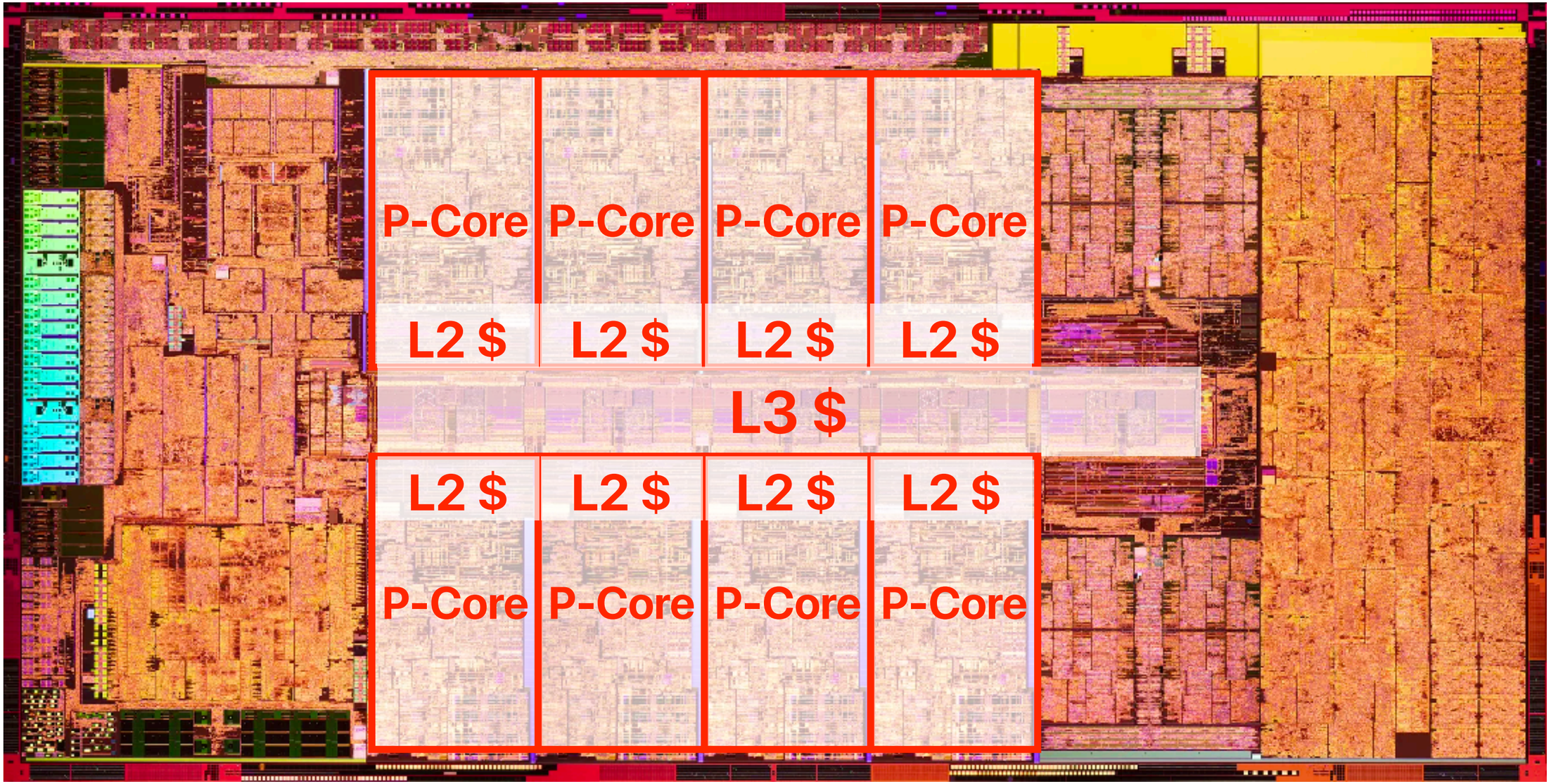
**There is no clear win on each — why not having both?**

```
Architecture:                    x86_64
CPU op-mode(s):                  32-bit, 64-bit
Byte Order:                      Little Endian
Address sizes:                   39 bits physical, 48 bits virtual
CPU(s):                          8
On-line CPU(s) list:             0-7
Thread(s) per core:              2
Core(s) per socket:              4
Socket(s):                       1
NUMA node(s):                    1
Vendor ID:                       GenuineIntel
CPU family:                      6
Model:                           151
Model name:                      12th Gen Intel(R) Core(TM) i3-12100F
Stepping:                        5
CPU MHz:                         3300.000
CPU max MHz:                     5500.0000
CPU min MHz:                     800.0000
BogoMIPS:                        6604.80
Virtualization:                  VT-x
L1d cache:                       192 KiB
L1i cache:                       128 KiB
```

# Modern processors have both CMP/SMT


AMD RyZen Processor
Core
L2 $
L3 $
Core
L2 $
Core
L2 $
Core
L2 $
Core
L2 $
L3 $
Core
L2 $
Core
L2 $
AMD
RYZEN

# Intel Alder Lake



# SMT v.s. CMP

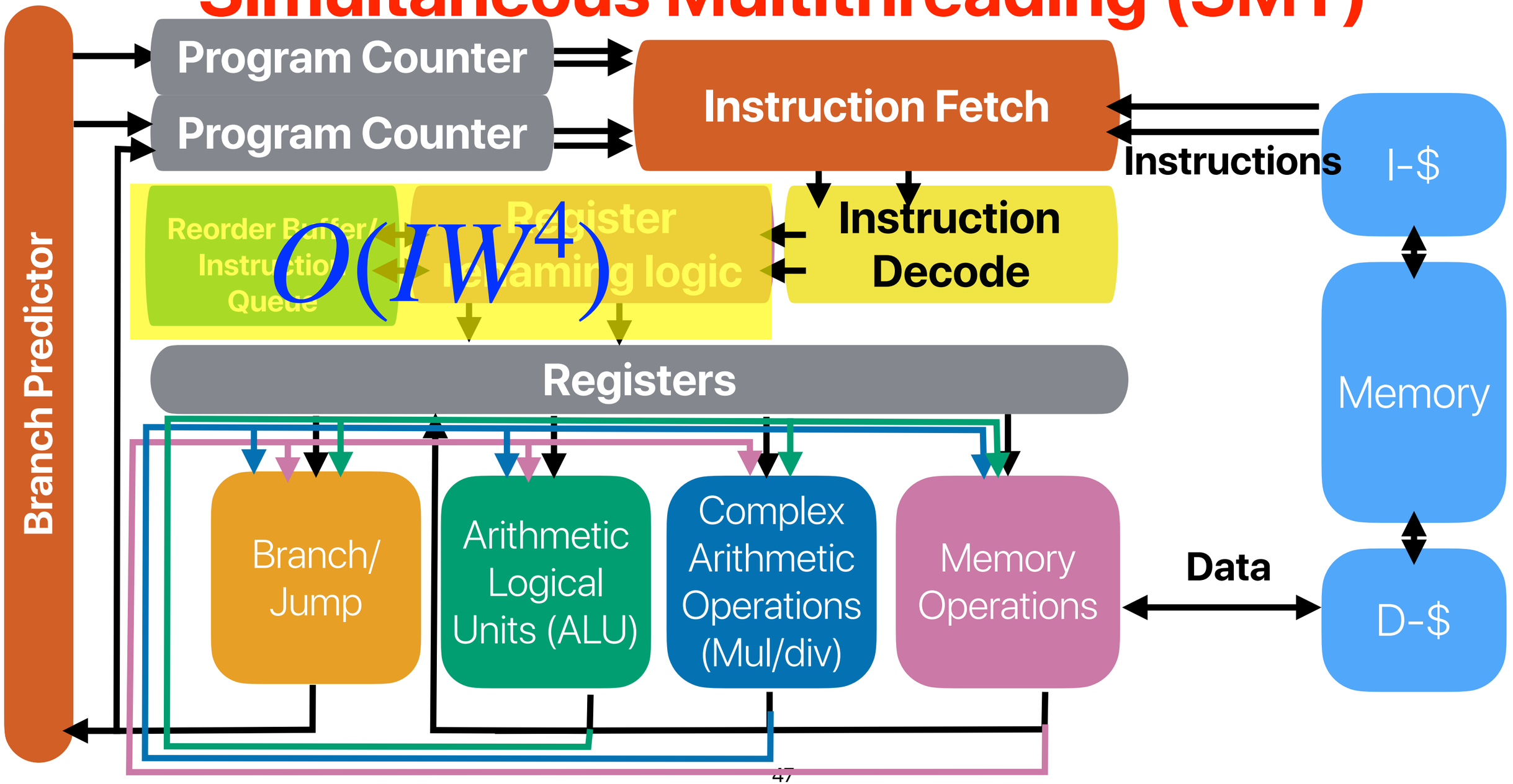
- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
  - ① If we are just running one program in the system, the program will perform better on an SMT processor — you have more resources for the program
  - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor — it depends!
  - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor — it depends!
  - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor — it depends!
  - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor — it depends!

  A. 1
  B. 2
  C. 3
  D. 4
  E. 5

**There is no clear win on each — why not having both?**

**The only thing we know for sure — if we don't parallel the program, it won't get any faster on SMT/CMP**

# Simultaneous Multithreading (SMT)



# Takeaways: parallel architectures

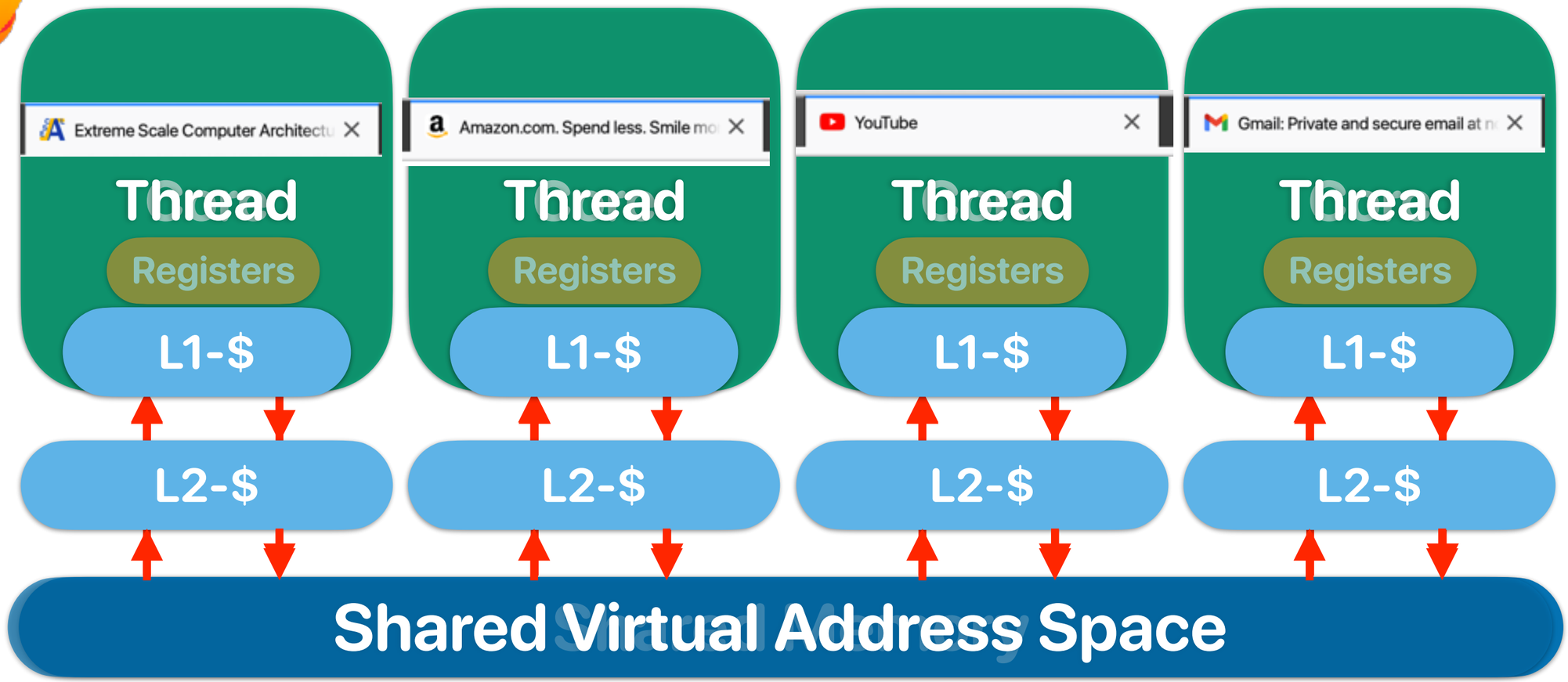
- SMT processors can better utilize the pipeline resources by allowing simultaneous execution of multiple threads
  - Improved execution throughput
  - May hurt the latency of each thread since we share functional units & cache
- CMP provides more processor cores for parallel threads or multiprogrammed environments
  - More isolated, protected pipeline
  - No flexibility if the running program needs more resource

# Parallel Programming & Architectural Supports for Parallel Programming

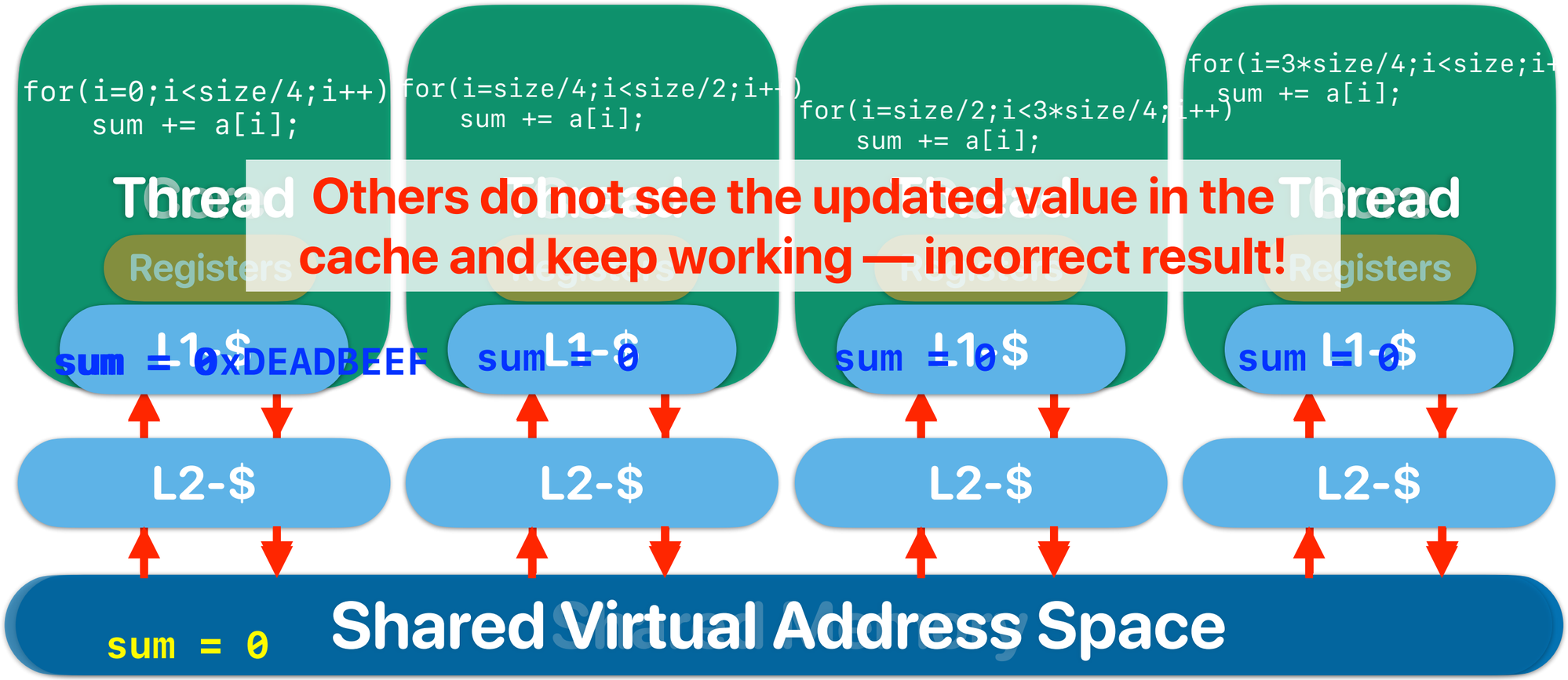
# Parallel programming

- To exploit parallelism you need to break your computation into multiple “processes” or multiple “threads”
- Processes (in OS/software systems)
  - Separate programs actually running (not sitting idle) on your computer at the same time.
  - Each process will have its own virtual memory space and you need explicitly exchange data using inter-process communication APIs
  - Programming model: MPI, Spark
- Threads (in OS/software systems)
  - Independent portions of your program that can run in parallel
  - All threads share the same virtual memory space
  - Programming model: pthread or openmp
- We will refer to these collectively as “threads”
  - A typical user system might have 1-8 actively running threads.
  - Servers can have more if needed (the sysadmins will hopefully configure it that way)

# What software thinks about “multiprogramming” hardware



# What software thinks about “multiprogramming” hardware




Others do not see the updated value in the cache and keep working — incorrect result!

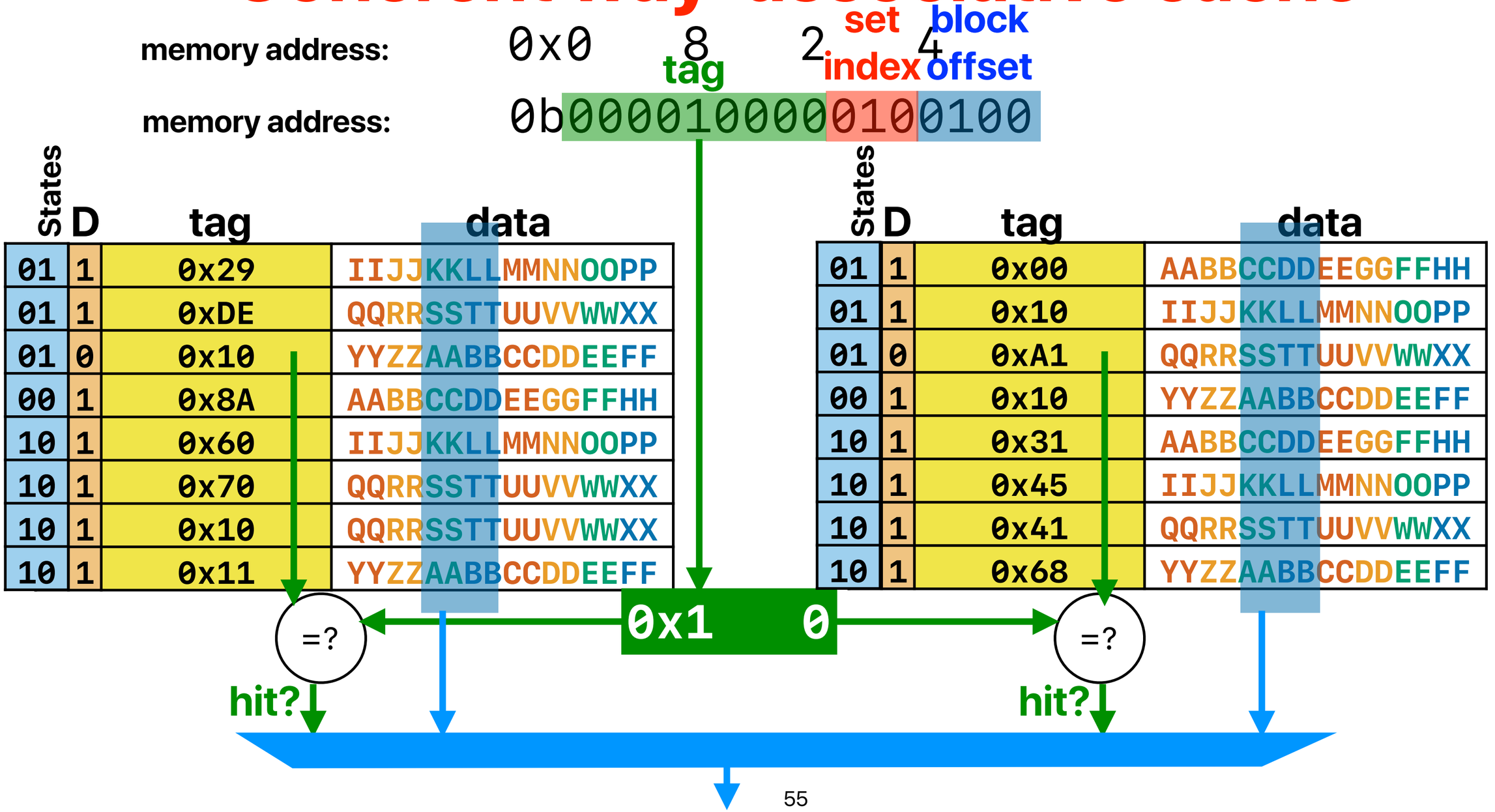
# Coherency & Consistency

- Coherency — Guarantees all processors see the same value for a variable/memory address in the system when the processors need the value at the same time
  - What value should be seen
- Consistency — All threads see the change of data in the same order
  - When the memory operation should be done

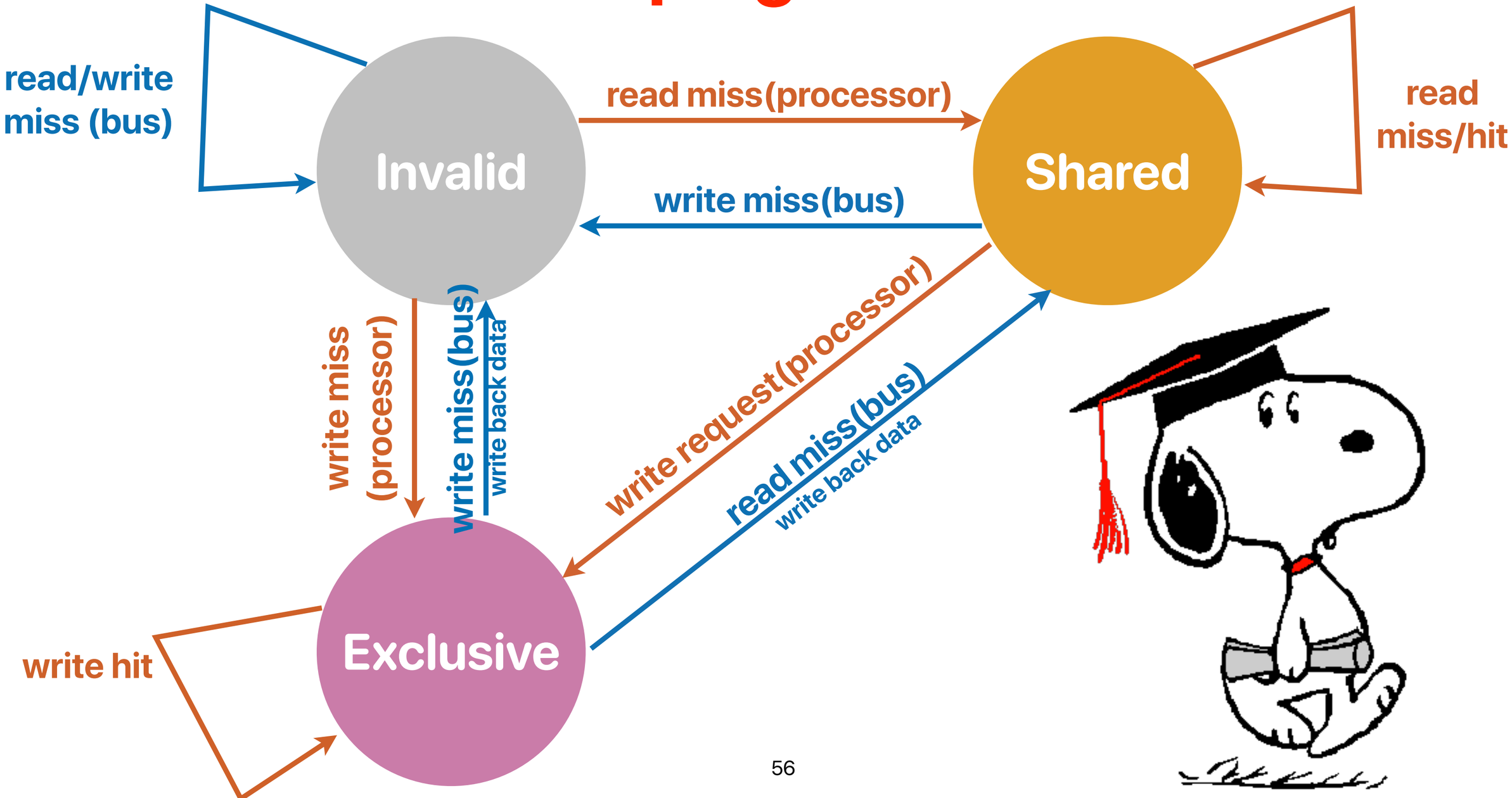
# Simple cache coherency protocol

- Snooping protocol
  - Each processor broadcasts / listens to cache misses
- State associate with each block (cacheline)
  - Invalid
    - The data in the current block is invalid
  - Shared
    - The processor can read the data
    - The data may also exist on other processors
  - Exclusive
    - The processor has full permission on the data
    - The processor is the only one that has up-to-date data

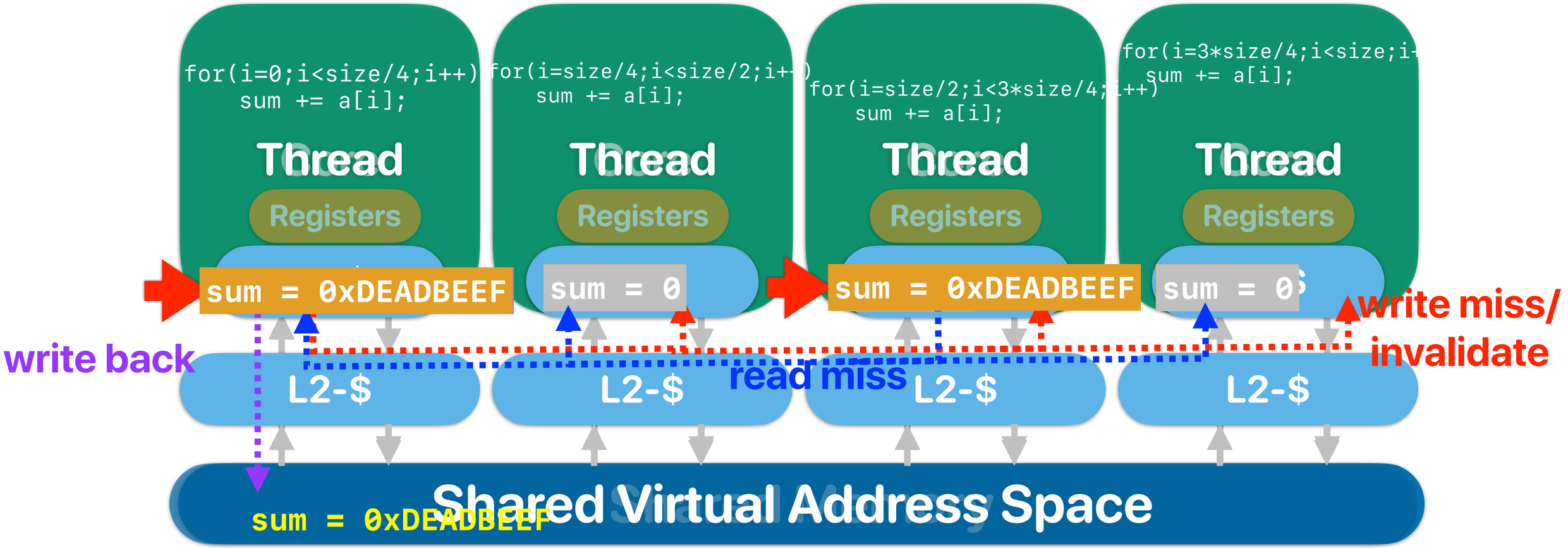
# Coherent way-associative cache



# Snooping Protocol



# What happens when we write in coherent caches?



# Observer

**thread 1**

```
int loop;

int main()
{
  pthread_t thread;
  loop = 1;

  pthread_create(&thread, NULL,
modifyloop, NULL);
  while(loop == 1)
  {
    continue;
  }
  pthread_join(thread, NULL);
  fprintf(stderr,"User input: %d\n",
```

**thread 2**

```
void* modifyloop(void *x)
{
  sleep(1);
  printf("Please input a number:\n");
  scanf("%d",&loop);
  return NULL;
}
```

# Observer

prevents the compiler from putting the variable “loop" in the “register”

**thread 1**

```
volatile int loop;

int main()
{
  pthread_t thread;
  loop = 1;

  pthread_create(&thread, NULL,
modifyloop, NULL);
  while(loop == 1)
  {
    continue;
  }
  pthread_join(thread, NULL);
  fprintf(stderr,"User input: %d\n",
```

**thread 2**

```
void* modifyloop(void *x)
{
  sleep(1);
  printf("Please input a number:\n");
  scanf("%d",&loop);
  return NULL;
}
```

# Announcements

- **Reading Quiz 8 (last reading quiz)** due **next Tuesday** before the lecture
- **Assignment 4** is **released** due this Saturday
- **Assignment 5** is **released** due 9/5/2024

Computer
Science &
Engineering

203

つづく